

Bore CLI Distribution & Installation Strategy

Overview

The Bore CLI is the client-side agent responsible for connecting a developer's local application to the Bore.ng tunneling platform. Users install the CLI, authenticate with their Bore account, and establish secure tunnels between the public internet and their local development environment.

Example usage:

```
bore login  
bore start myapp --port 3000
```

Distribution Methods

There are three primary ways to distribute the Bore CLI:

1. Python Package (PyPI)
2. Standalone Executables
3. GitHub Releases

A professional implementation can support all three methods.

Method 1: Publish as a Python Package

Advantages

- Fastest to implement
- Easy updates
- Familiar workflow for Python developers
- Supports Windows, Linux, and macOS

Installation

```
pip install bore-cli
```

Project Structure

```
bore-cli/  
├── bore/  
│   ├── __init__.py  
│   ├── cli.py  
│   ├── auth.py  
│   ├── tunnel.py  
│   ├── websocket_client.py  
│   └── config.py  
├── pyproject.toml  
├── README.md  
└── LICENSE
```

Build Package

Install build tools:

```
pip install build twine
```

Build:

```
python -m build
```

Output:

```
dist/  
├── bore_cli-0.1.0.tar.gz  
└── bore_cli-0.1.0-py3-none-any.whl
```

Test Locally

```
pip install dist/bore_cli-0.1.0-py3-none-any.whl
```

Verify:

```
bore --help
```

Publish to PyPI

Create a PyPI account and API token.

Upload:

```
twine upload dist/*
```

Users can then install using:

```
pip install bore-cli
```

Method 2: Standalone Executables

Advantages

- No Python installation required
- Better user experience
- Simpler onboarding

Build with PyInstaller

Install:

```
pip install pyinstaller
```

Build:

```
pyinstaller --onefile bore/cli.py --name bore
```

Output:

```
dist/  
└─ bore.exe
```

Windows

```
bore.exe login
```

Linux

```
chmod +x bore  
./bore login
```

macOS

```
chmod +x bore  
./bore login
```

Method 3: GitHub Releases

Advantages

- Easy distribution
- Versioned releases
- Download analytics
- Community-friendly

Repository Structure:

```
github.com/bore-ng/bore-cli
```

Release Assets:

```
bore-windows-amd64.exe  
bore-linux-amd64  
bore-darwin-arm64
```

Users download directly from GitHub Releases.

Linux example:

```
chmod +x bore-linux-amd64  
sudo mv bore-linux-amd64 /usr/local/bin/bore
```

Verify:

```
bore --help
```

Recommended CLI Commands

Authentication

Login:

```
bore login
```

Logout:

```
bore logout
```

Current Account:

```
bore whoami
```

Tunnel Management

Start Tunnel:

```
bore start myapp --port 3000
```

Stop Tunnel:

```
bore stop myapp
```

List Tunnels:

```
bore tunnels
```

Tunnel Status:

```
bore status myapp
```

CLI Authentication Flow

User logs in:

```
bore login
```

CLI sends:

```
POST /api/login/
```

Response:

```
{  
  "access": "jwt-token"  
}
```

JWT is stored locally:

```
~/.bore/config.json
```

Example:

```
{  
  "token": "jwt-token"  
}
```

Tunnel Creation Workflow

Step 1

User creates tunnel in dashboard:

```
myapp.bore.ng
```

Status:

```
Offline
```

Step 2

User starts tunnel:

```
bore start myapp --port 3000
```

Step 3

CLI authenticates and requests tunnel activation:

```
POST /api/tunnels/start/
```

Step 4

Backend responds:

```
{
  "url": "https://myapp.bore.ng",
  "ws_url": "wss://api.bore.ng/ws/tunnel/myapp/"
}
```

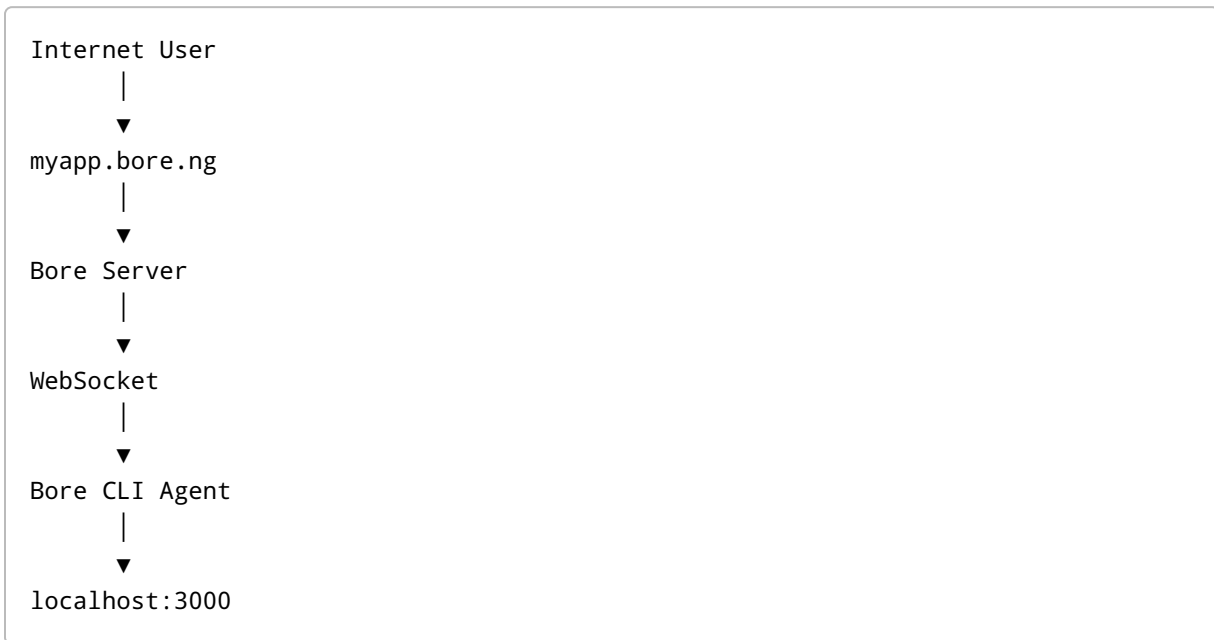
Step 5

CLI opens WebSocket connection.

Step 6

Traffic begins forwarding.

WebSocket Tunnel Flow



The agent maintains a persistent WebSocket connection and forwards requests to the user's local application.

MVP Roadmap

Phase 1

- User Registration
- User Login
- Dashboard
- Tunnel Model
- Tunnel Creation

Phase 2

- Bore CLI Login
- Bore CLI Start Tunnel
- JWT Authentication
- WebSocket Connectivity

Phase 3

- HTTP Request Forwarding

- HTTPS Support
- Tunnel Logs
- Request Replay

Phase 4

- Multiple Tunnels
 - Team Accounts
 - Analytics
 - Custom Domains
-

Recommended Initial Distribution Strategy

For the first Bore.ng release:

1. Publish to PyPI
2. Build Windows executable
3. Host source code on GitHub
4. Add GitHub Actions for automated builds

Installation options:

```
pip install bore-cli
```

or

```
Download bore.exe from GitHub Releases
```

This provides the fastest path to onboarding early testers while keeping maintenance overhead low.